

```
In [18]: dill.load_session('cars 28.10.pkl')
print("Session restored 
```

Session restored 

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import numpy as np
import sklearn as sk
import category_encoders
import dill
```

```
In [3]: from io import StringIO
def save_slide(name):
    plt.savefig(f"{name}.png")
```

```
In [4]: import os
print(os.getcwd())
```

C:\Users\adham\anaconda_projects\used cars

```
In [119... car_data = pd.read_csv("car data.csv")
car_data.head()
```

```
Out[119...
```

	Name	Price	Color	Mileage	Make	Model	City	Date Displayed	Aut Transi
0	Kia Sportage 2024	2800000	Dark grey	300.0	Kia	Sportage	Tagamo3 - New Cairo	2/16/2024	
1	Fiat Tipo 2021	1200000	Petroleum	131000.0	Fiat	Tipo	Kafr el-Dawwar	2/16/2024	
2	Hyundai Tucson Turbo GDI 2021	2050000	Black	125000.0	Hyundai	Tucson Turbo GDI	Nasr city	2/16/2024	
3	BMW X1 2018	2050000	Gray	84000.0	BMW	X1	Cairo	2/16/2024	
4	Chevrolet Optra 2021	700000	Blue	75000.0	Chevrolet	Optra	Obour City	2/16/2024	

In [94]: `car_data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32625 entries, 0 to 32624
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Name                                  32625 non-null  object
1   Price                                32625 non-null  int64
2   Color                                32625 non-null  object
3   Mileage                              32625 non-null  float64
4   Make                                  32625 non-null  object
5   Model                                32625 non-null  object
6   City                                  32625 non-null  object
7   Date Displayed                       32625 non-null  object
8   Automatic Transmission                32625 non-null  int64
9   Air Conditioner                      32625 non-null  int64
10  Power Steering                       32625 non-null  int64
11  Remote Control                       32625 non-null  int64
12  Item URL                              32625 non-null  object
dtypes: float64(1), int64(5), object(7)
memory usage: 3.2+ MB
```

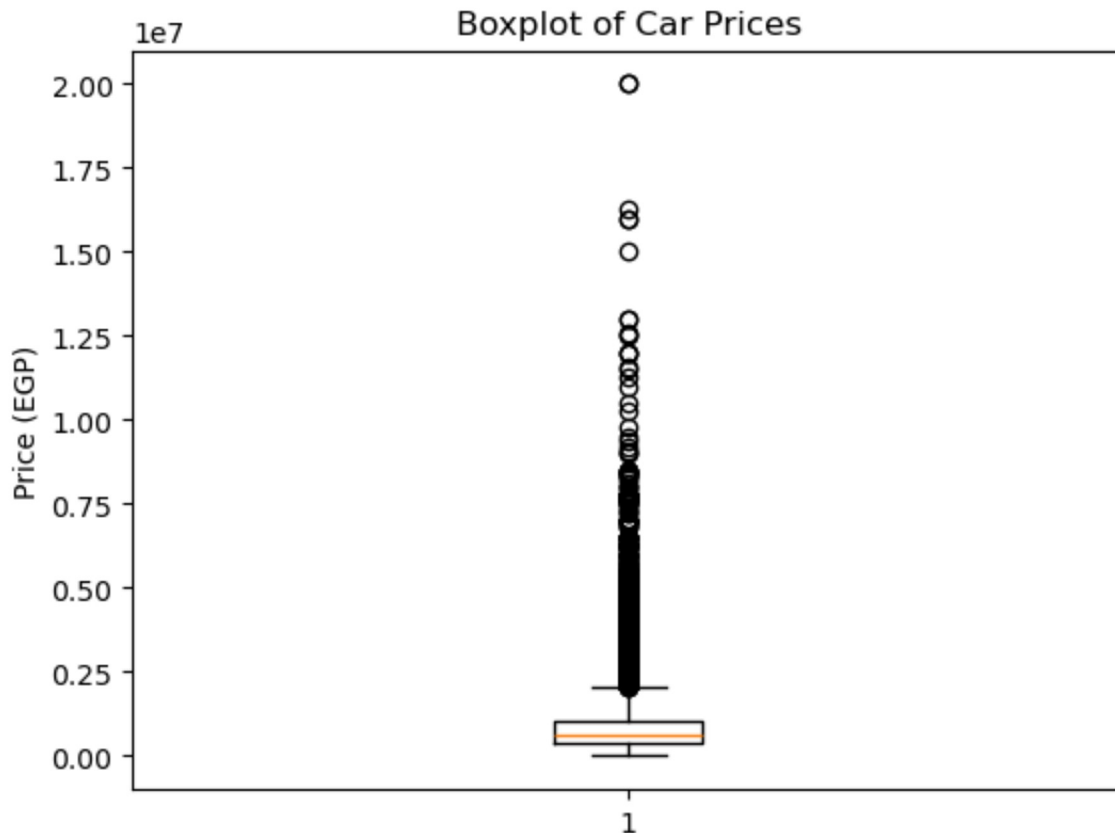
In [95]: `car_data.describe()`

Out[95]:

	Price	Mileage	Automatic Transmission	Air Conditioner	Power Steering	Remote Control
count	3.262500e+04	3.262500e+04	32625.000000	32625.000000	32625.000000	32625.000000
mean	9.544847e+05	1.511668e+05	0.642299	0.615418	0.555739	0.573119
std	1.086724e+06	2.123402e+05	0.479331	0.486504	0.496891	0.494632
min	1.060000e+03	0.000000e+00	0.000000	0.000000	0.000000	0.000000
25%	3.830000e+05	7.000000e+04	0.000000	0.000000	0.000000	0.000000
50%	6.500000e+05	1.310000e+05	1.000000	1.000000	1.000000	1.000000
75%	1.050000e+06	1.850000e+05	1.000000	1.000000	1.000000	1.000000
max	2.000000e+07	2.226000e+06	1.000000	1.000000	1.000000	1.000000

Plotting a Boxplot to see if there's any extreme outliers

```
In [13]: plt.boxplot(car_data['Price'])
plt.title("Boxplot of Car Prices")
plt.ylabel("Price (EGP)")
plt.savefig("Boxp before.png")
plt.show()
```



removing the extreme outliers

```
In [30]: Q1 = car_data['Price'].quantile(0.25)
Q3 = car_data['Price'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

car_data = car_data[(car_data['Price'] >= lower) & (car_data['Price'] <= upper)]
```

```
In [31]: plt.boxplot(car_data['Price'])
plt.title("Boxplot of Car Prices")
plt.ylabel("Price (EGP)")
plt.savefig("Boxp after.png")
plt.show()
```



In [99]: `car_data.describe()`

Out[99]:

	Price	Mileage	Automatic Transmission	Air Conditioner	Power Steering	Remote Control
count	2.969200e+04	2.969200e+04	29692.000000	29692.000000	29692.000000	29692.000000
mean	7.012785e+05	1.602433e+05	0.614745	0.600633	0.535902	0.553445
std	4.594187e+05	2.195449e+05	0.486664	0.489777	0.498718	0.497143
min	1.060000e+03	0.000000e+00	0.000000	0.000000	0.000000	0.000000
25%	3.500000e+05	8.000000e+04	0.000000	0.000000	0.000000	0.000000
50%	6.075000e+05	1.360000e+05	1.000000	1.000000	1.000000	1.000000
75%	9.000000e+05	1.956100e+05	1.000000	1.000000	1.000000	1.000000
max	2.050000e+06	2.226000e+06	1.000000	1.000000	1.000000	1.000000

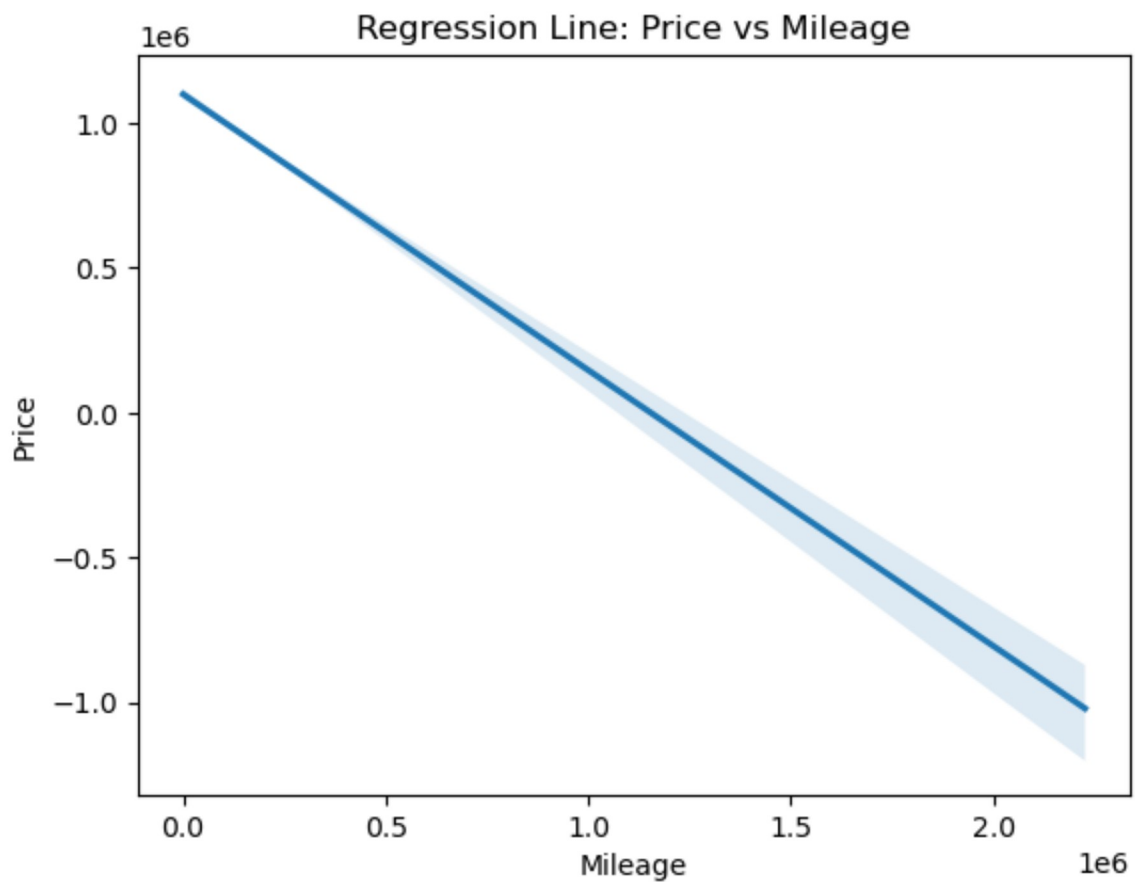
Relationships between attributes

In [100]: `car_data[['Price', 'Mileage']].corr()`

Out[100...

	Price	Mileage
Price	1.000000	-0.182069
Mileage	-0.182069	1.000000

```
In [15]: sns.regplot(x='Mileage', y='Price', data=car_data, scatter=False)
plt.title("Regression Line: Price vs Mileage")
plt.xlabel("Mileage")
plt.ylabel("Price")
plt.savefig("price mil.png")
plt.show()
```



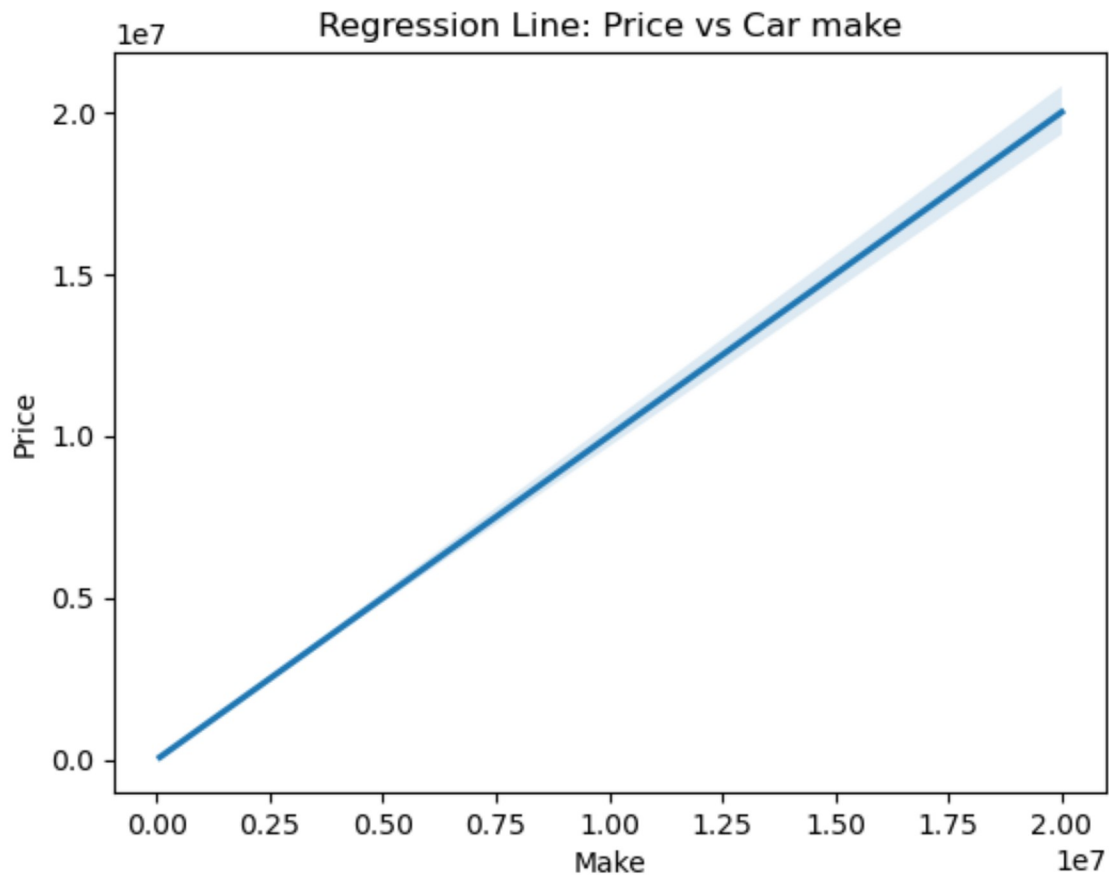
Encoding the "Make" column to see if there's any correlation between the brand and the price

```
In [20]: car_data['Make_encoded'] = car_data.groupby('Make')['Price'].transform('mean')
car_data[['Price', 'Make_encoded', 'Mileage']].corr()
```

```
Out[20]:
```

	Price	Make_encoded	Mileage
Price	1.000000	0.632802	-0.186006
Make_encoded	0.632802	1.000000	-0.070176
Mileage	-0.186006	-0.070176	1.000000

```
In [21]: sns.regplot(x='Make_encoded', y='Price', data=car_data, scatter=False)
plt.title("Regression Line: Price vs Car make")
plt.xlabel("Make")
plt.ylabel("Price")
plt.savefig("price model.png")
plt.show()
```

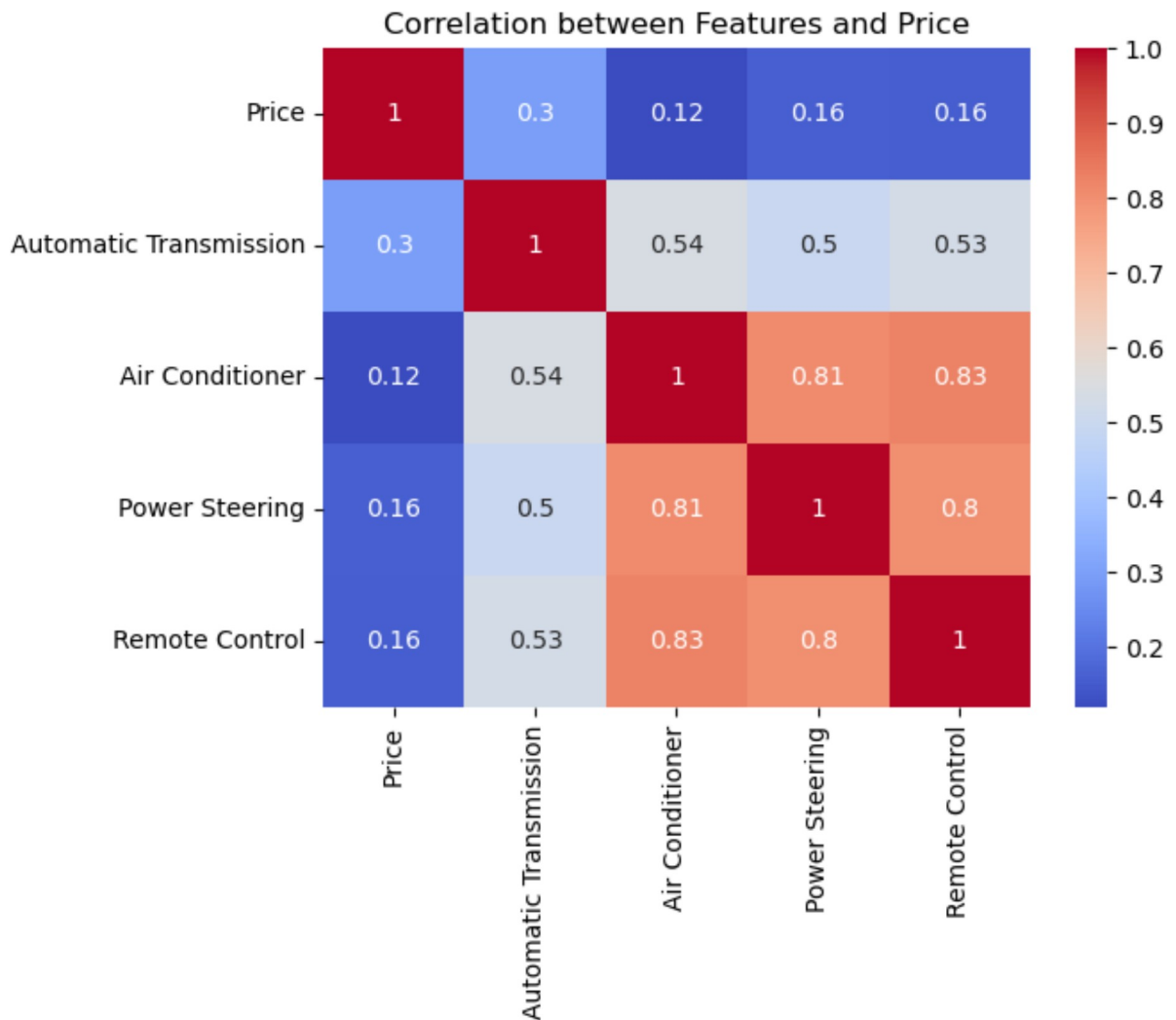


A correlation graph showing the relationship between the price and the other Yes/No car features

```
In [23]: corr = car_data[['Price', 'Automatic Transmission', 'Air Conditioner', 'Power Steer

sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation between Features and Price')
plt.savefig("yes no corr.png")
```

```
plt.show()
```



Preparing a new column combining the car's brand and model for the to encode it later for the ML process

```
In [120...] car_data['brand_model'] = car_data['Make'] + ' ' + car_data['Model']
```

```
In [6]: car_data.to_csv("car data final.csv")
```

choosing the relevant features for the ML process

```
In [121...] X = car_data[['Mileage', 'Automatic Transmission', 'Air Conditioner', 'Power Steering']
Y = car_data['Price']
from sklearn.preprocessing import OneHotEncoder
```

In [122... `X.head()`

Out[122...

	Mileage	Automatic Transmission	Air Conditioner	Power Steering	Remote Control	brand_model
0	300.0	1	1	1	1	Kia Sportage
1	131000.0	1	1	1	1	Fiat Tipo
2	125000.0	1	1	1	1	Hyundai Tucson Turbo GDI
3	84000.0	1	1	1	1	BMW X1
4	75000.0	1	1	1	1	Chevrolet Optra

Encoding the brand_model column

In [123... `car_data['brand_model'] = car_data.groupby('Make')['Price'].transform('mean')`
`car_data[['Price', 'brand_model', 'Mileage']].corr()`

Out[123...

	Price	brand_model	Mileage
Price	1.000000	0.632802	-0.186006
brand_model	0.632802	1.000000	-0.070176
Mileage	-0.186006	-0.070176	1.000000

Starting with the Random Forest model using pipeline to prevent data leakage

In [124... `from sklearn.model_selection import train_test_split`
`from category_encoders import TargetEncoder`
`from sklearn.pipeline import Pipeline`

`X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)`

`rf_pipeline = Pipeline([`
 `('target_encode', TargetEncoder(cols=['brand_model'])),`
 `('model', RandomForestRegressor(random_state=42))`
`])`

`rf_pipeline.fit(X_train, Y_train)`

`Y_pred = rf_pipeline.predict(X_test)`

Later on, I decided to add other models to compare the R2, MAE and RMSE

```
In [125... pipelines = {
    'Linear': Pipeline([
        ('encode', TargetEncoder(cols=['brand_model'])),
        ('model', LinearRegression())
    ]),
    'Decision Tree': Pipeline([
        ('encode', TargetEncoder(cols=['brand_model'])),
        ('model', DecisionTreeRegressor(random_state=42))
    ]),
    'Random Forest': rf_pipeline
}

for name, pipe in pipelines.items():
    pipe.fit(X_train, Y_train)
    preds = pipe.predict(X_test)
    r2 = r2_score(Y_test, preds)
    mae = mean_absolute_error(Y_test, preds)
    rmse = np.sqrt(mean_squared_error(Y_test, preds))
    print(f"{name}:")
    print(f"R2: {r2}")
    print(f"MAE: {mae}")
    print(f"RMSE: {rmse}")
    print("-" * 40)
```

Linear:

R2: 0.7289121172180641
MAE: 275216.99981155916
RMSE: 531550.6554443578

Decision Tree:

R2: 0.8092435218700269
MAE: 177063.32617966368
RMSE: 445891.5610427017

Random Forest:

R2: 0.8448630328752663
MAE: 153023.82964063206
RMSE: 402112.2035982197

Graphing the actual and predicted prices

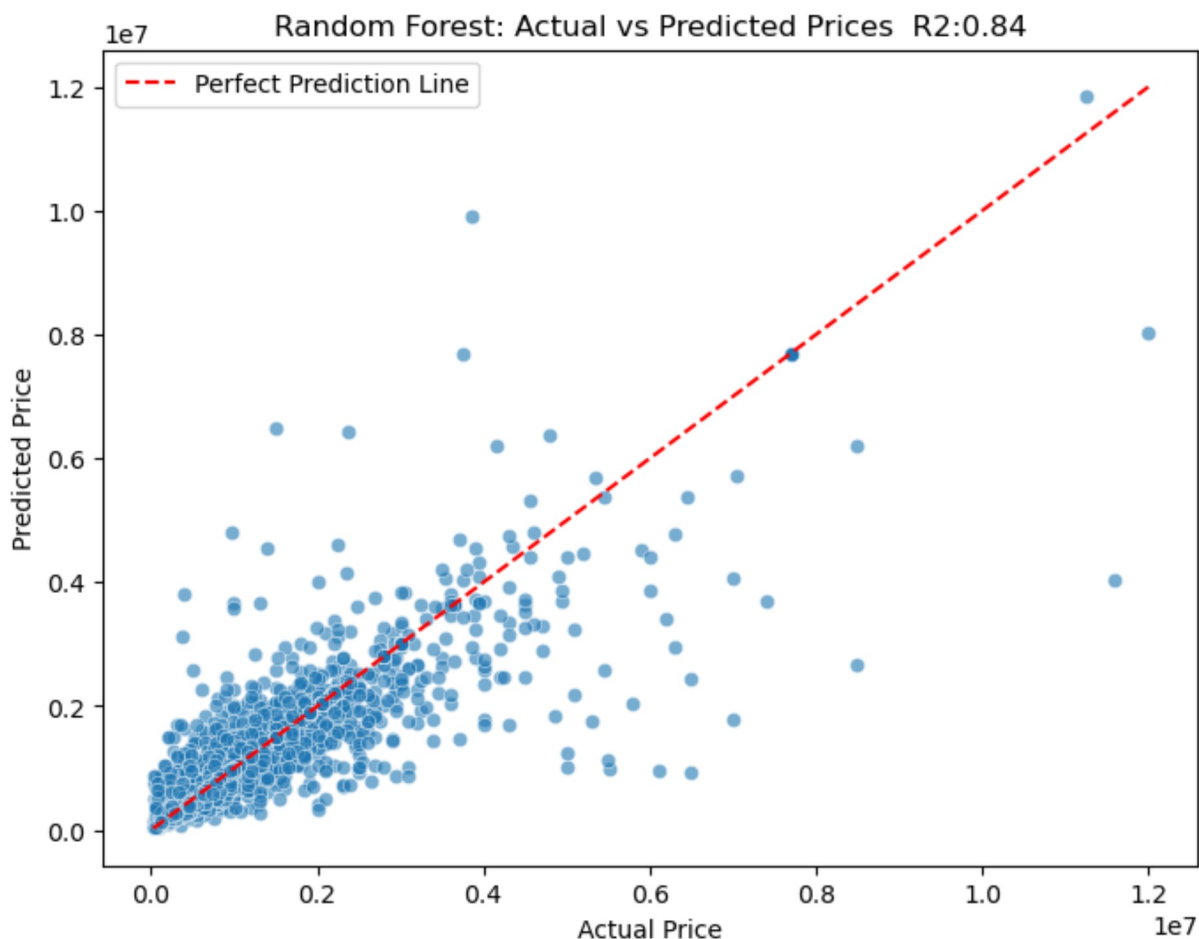
```
In [24]: Y_pred = rf_pipeline.predict(X_test)

plt.figure(figsize=(8, 6))
sns.scatterplot(x=Y_test, y=Y_pred, alpha=0.6)
plt.plot([Y_test.min(), Y_test.max()],
```

```

[Y_test.min(), Y_test.max()],
color='red', linestyle='--', label='Perfect Prediction Line')
plt.title("Random Forest: Actual vs Predicted Prices R2:0.84")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.legend()
plt.savefig("rf before cv.png")
plt.show()

```

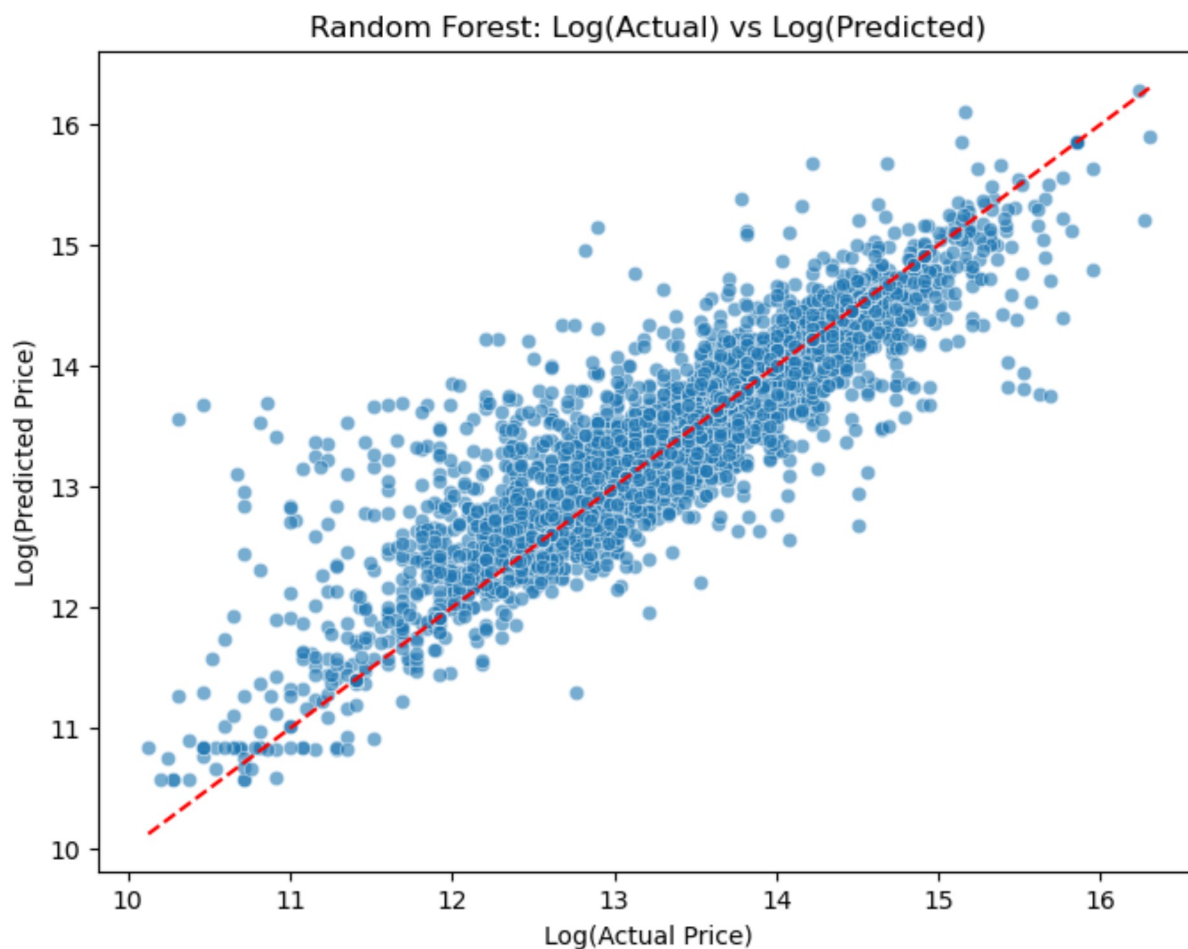


Log transforming the price since car prices are HIGHLY skewed

```

In [25]: plt.figure(figsize=(8,6))
sns.scatterplot(x=np.log1p(Y_test), y=np.log1p(Y_pred), alpha=0.6)
plt.plot([np.log1p(Y_test).min(), np.log1p(Y_test).max()],
         [np.log1p(Y_test).min(), np.log1p(Y_test).max()],
         color='red', linestyle='--')
plt.title("Random Forest: Log(Actual) vs Log(Predicted)")
plt.xlabel("Log(Actual Price)")
plt.ylabel("Log(Predicted Price)")
plt.savefig("rf log price.png")
plt.show()

```

Graphing feature importance to see which feature truly affects the price

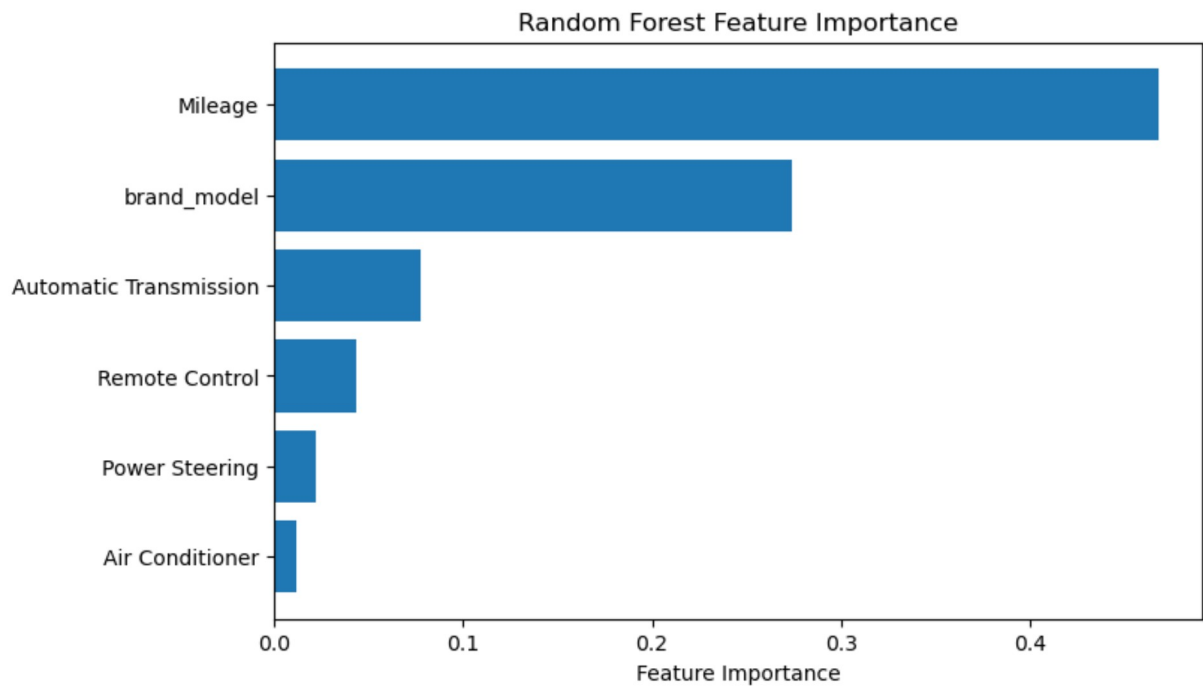
```
In [26]: features = ['Mileage', 'Automatic Transmission', 'Air Conditioner',
                    'Power Steering', 'Remote Control', 'brand_model']

importances = rf.feature_importances_

if len(importances) > len(features):
    importances = importances[:len(features)]

indices = np.argsort(importances)[::-1]

plt.figure(figsize=(8,5))
plt.barh(np.array(features)[indices], importances[indices])
plt.xlabel("Feature Importance")
plt.title("Random Forest Feature Importance")
plt.gca().invert_yaxis()
plt.savefig("feature_imp.png")
plt.show()
```



Trying to cross-validate the model to make sure that there was no leakage

```
In [34]: from sklearn.model_selection import GridSearchCV

rf_pipeline = Pipeline([
    ('encode', TargetEncoder(cols=['brand_model'])),
    ('model', RandomForestRegressor(random_state=42))
])

param_grid = {
    'model__n_estimators': [100, 200, 500],
    'model__max_depth': [10, 20, None],
    'model__min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(
    rf_pipeline,
    param_grid,
    cv=3,
    scoring='r2',
    n_jobs=-1
)

grid.fit(X_train, Y_train)

print("Best Parameters:", grid.best_params_)
print("Best Cross-Validated R2:", grid.best_score_)
```


Best Parameters: {'model__max_depth': 20, 'model__min_samples_split': 5, 'model__n_estimators': 500}

Best Cross-Validated R²: 0.8188511815092946

```
In [35]: best_pipeline = grid.best_estimator_
best_pipeline.fit(X_train, Y_train)
Y_pred = best_pipeline.predict(X_test)
r2 = r2_score(Y_test, Y_pred)
mae = mean_absolute_error(Y_test, Y_pred)
rmse = np.sqrt(mean_squared_error(Y_test, Y_pred))

print("Cross-validated Random Forest scores:")
print(f"R2: {r2}")
print(f"MAE: {mae}")
print(f"RMSE: {rmse}")
```

Cross-validated Random Forest scores:

R2: 0.8489498268002008

MAE: 149638.07648346506

RMSE: 396780.40734543354

Got almost the same result which is a good sign, no trace of leakage or other distortions

Saving the chosen model and creating a simple input/output program to demonstrate the model's results

```
In [136... import joblib
joblib.dump(best_pipeline, "final_rf_model.pkl")
```

```
Out[136... ['final_rf_model.pkl']
```

```
In [140... expected_features = ['Mileage', 'Automatic Transmission', 'Air Conditioner',
                        'Power Steering', 'Remote Control', 'brand_model']

print("Enter car details:")
mileage = float(input("Mileage (in km): "))
auto = int(input("Automatic Transmission (1 for Yes, 0 for No): "))
ac = int(input("Air Conditioner (1 for Yes, 0 for No): "))
ps = int(input("Power Steering (1 for Yes, 0 for No): "))
remote = int(input("Remote Control (1 for Yes, 0 for No): "))
brand_model = input("Brand and Model (e.g., Toyota Corolla): ")

data = pd.DataFrame([{'Mileage': mileage,
                      'Automatic Transmission': auto,
                      'Air Conditioner': ac,
```

```

        'Power Steering': ps,
        'Remote Control': remote,
        'brand_model': brand_model
    })

prediction = grid.predict(data)[0]

print(f"\nEstimated Price: {prediction:,.0f} EGP")

```

Enter car details:

Estimated Price: 699,742 EGP

Took it up a notch and created a UI to make the program more appealing using streamlit

```

In [11]: import streamlit as st
import pandas as pd
import numpy as np
import joblib

from sklearn.pipeline import Pipeline
from category_encoders import TargetEncoder
from sklearn.ensemble import RandomForestRegressor

rf_pipeline = joblib.load("final_rf_model.pkl")

st.set_page_config(page_title="Used Car Price Predictor", layout="centered")

st.title("Used Car Price Prediction App")
st.markdown("Enter car details below to estimate its price.")

mileage = st.number_input("Mileage (in km):", min_value=0, max_value=2_000_000, val
auto = st.selectbox("Automatic Transmission:", [0, 1], format_func=lambda x: "Yes"
ac = st.selectbox("Air Conditioner:", [0, 1], format_func=lambda x: "Yes" if x == 1
ps = st.selectbox("Power Steering:", [0, 1], format_func=lambda x: "Yes" if x == 1
remote = st.selectbox("Remote Control:", [0, 1], format_func=lambda x: "Yes" if x ==
brand_model = st.text_input("Brand and Model (e.g., Toyota Corolla):")

if st.button("Predict Price"):
    # Make DataFrame
    input_data = pd.DataFrame([
        'Mileage': mileage,
        'Automatic Transmission': auto,
        'Air Conditioner': ac,
        'Power Steering': ps,
        'Remote Control': remote,
        'brand_model': brand_model
    ])

    try:
        prediction = rf_pipeline.predict(input_data)[0]

```

```
st.success(f"**Estimated Price:** {prediction:,.0f} EGP")  
  
except Exception as e:  
    st.error(f"Error during prediction: {e}")
```

[illegible]

```
2025-10-28 19:49:03.404 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.406 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.407 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.408 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.409 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.411 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.412 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.414 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.415 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.416 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.417 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.419 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.421 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.422 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.423 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.424 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.426 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
2025-10-28 19:49:03.427 Thread 'MainThread': missing ScriptRunContext! This warning
can be ignored when running in bare mode.
```

```
In [17]: dill.dump_session('cars 28.10.pkl')
         print("Session saved )
```

Session saved 